

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

df = pd.read_csv("customer_purchase_dataset.csv")

```

```
df
```

	Age	AnnualIncome	WebsiteVisits	Purchased
0	25	114130	7	1
1	63	109312	10	1
2	22	33944	8	0
3	43	62122	16	1
4	24	44479	2	0
..	...	...	...	...
195	62	26557	19	0
196	29	109753	4	1
197	32	93827	7	1
198	50	94393	10	1
199	45	62944	1	0

```
[200 rows x 4 columns]
```

```
df.head()
```

	Age	AnnualIncome	WebsiteVisits	Purchased
0	25	114130	7	1
1	63	109312	10	1
2	22	33944	8	0
3	43	62122	16	1
4	24	44479	2	0

```
df.tail()
```

	Age	AnnualIncome	WebsiteVisits	Purchased
195	62	26557	19	0
196	29	109753	4	1
197	32	93827	7	1
198	50	94393	10	1
199	45	62944	1	0

```
df.shape
```

```
(200, 4)
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 200 entries, 0 to 199
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	200 non-null	int64
1	AnnualIncome	200 non-null	int64
2	WebsiteVisits	200 non-null	int64
3	Purchased	200 non-null	int64

```
dtypes: int64(4)
```

```
memory usage: 6.4 KB
```

```
df.isnull().sum()
```

```
Age          0
AnnualIncome  0
WebsiteVisits 0
Purchased    0
dtype: int64
```

```
df.describe()
```

	Age	AnnualIncome	WebsiteVisits	Purchased
count	200.000000	200.000000	200.000000	200.000000
mean	40.910000	71968.28500	10.735000	0.660000
std	13.764783	29470.71412	5.905859	0.474898
min	18.000000	20685.00000	1.000000	0.000000
25%	30.000000	46016.75000	5.000000	0.000000
50%	39.500000	72220.50000	11.000000	1.000000
75%	51.000000	96874.50000	16.000000	1.000000
max	65.000000	119595.00000	20.000000	1.000000

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.corr()
```

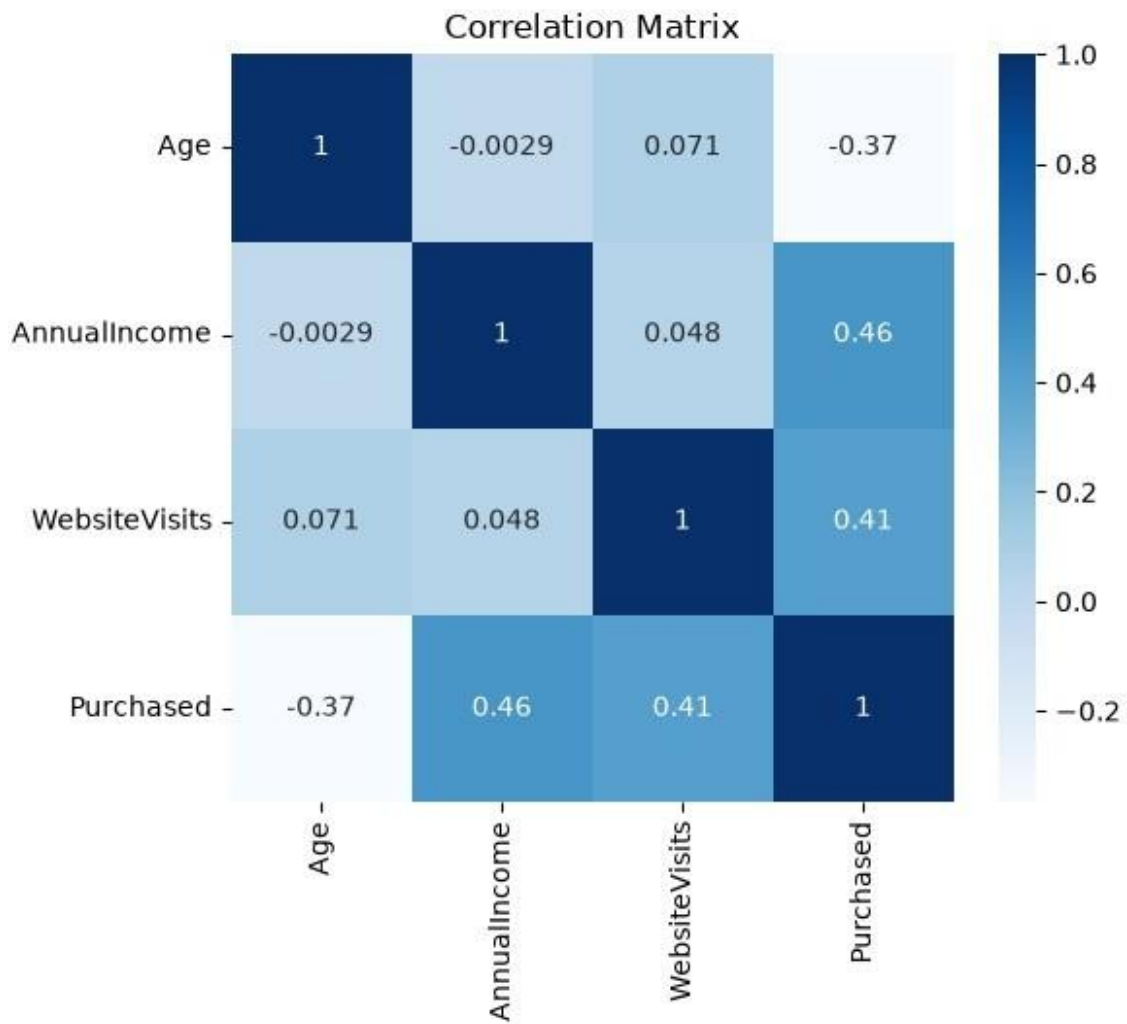
	Age	AnnualIncome	WebsiteVisits	Purchased
Age	1.000000	-0.002943	0.070916	-0.367548
AnnualIncome	-0.002943	1.000000	0.048473	0.462331
WebsiteVisits	0.070916	0.048473	1.000000	0.412054
Purchased	-0.367548	0.462331	0.412054	1.000000

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(df.corr(), annot=True, cmap='Blues')
```

```
plt.title("Correlation Matrix")
```

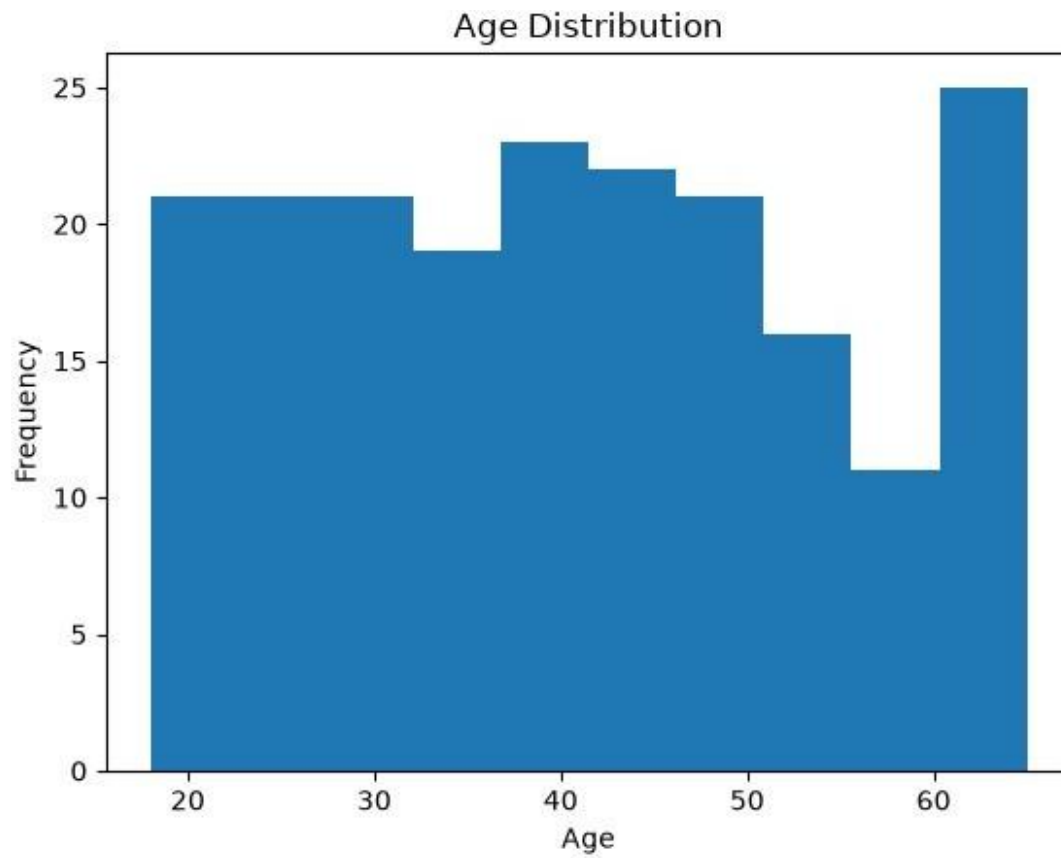
```
plt.show()
```



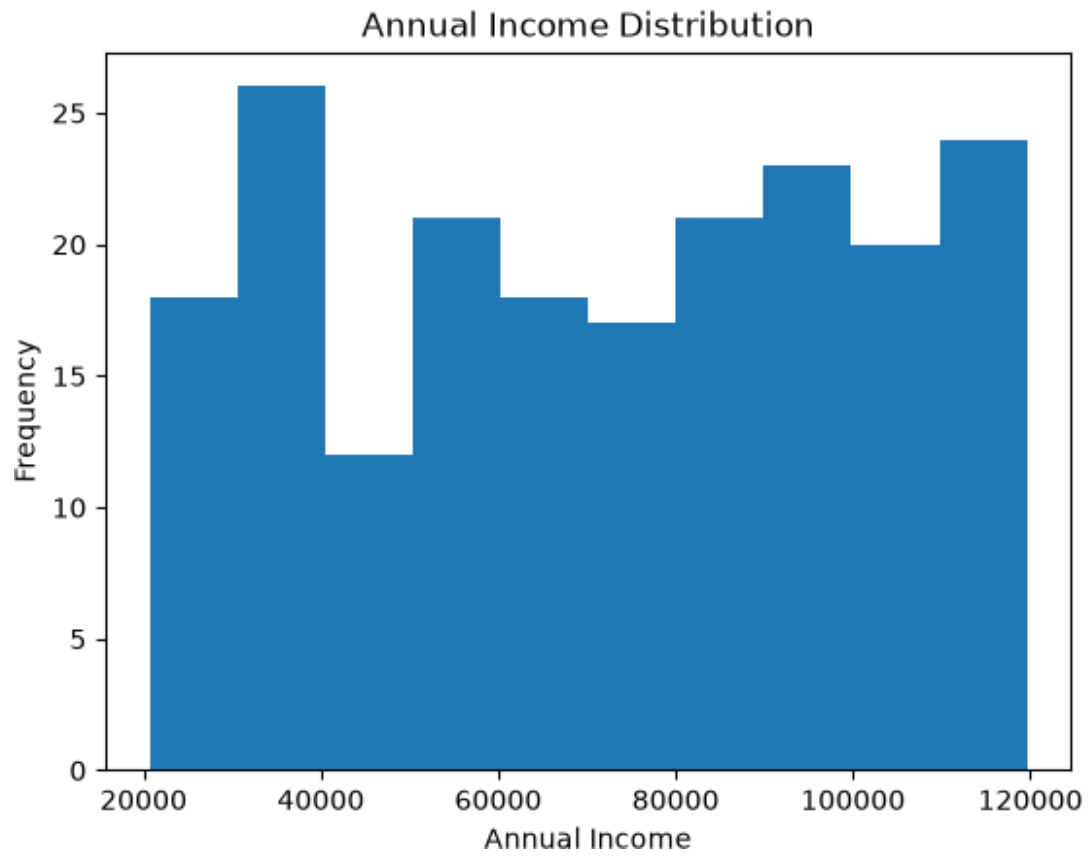
```
sns.countplot(x='Purchased', data=df)
plt.title("Customer Purchase Distribution")
plt.show()
```



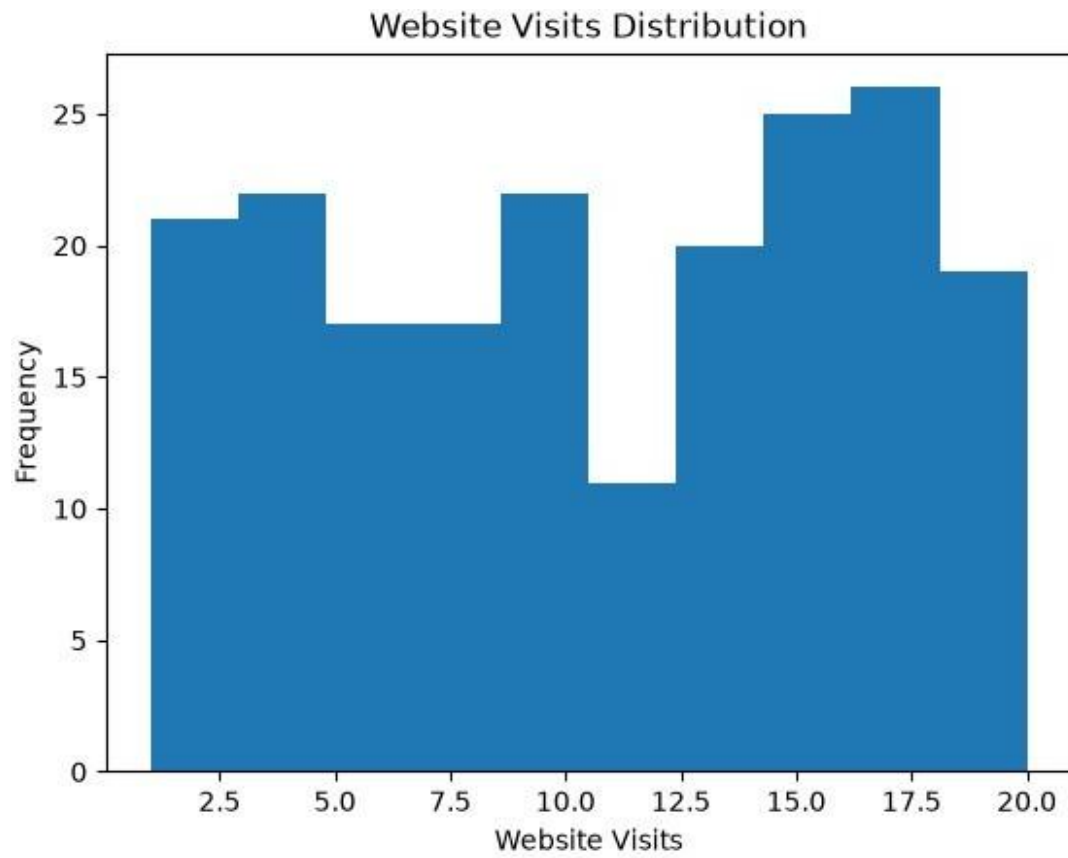
```
plt.hist(df['Age'], bins=10)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



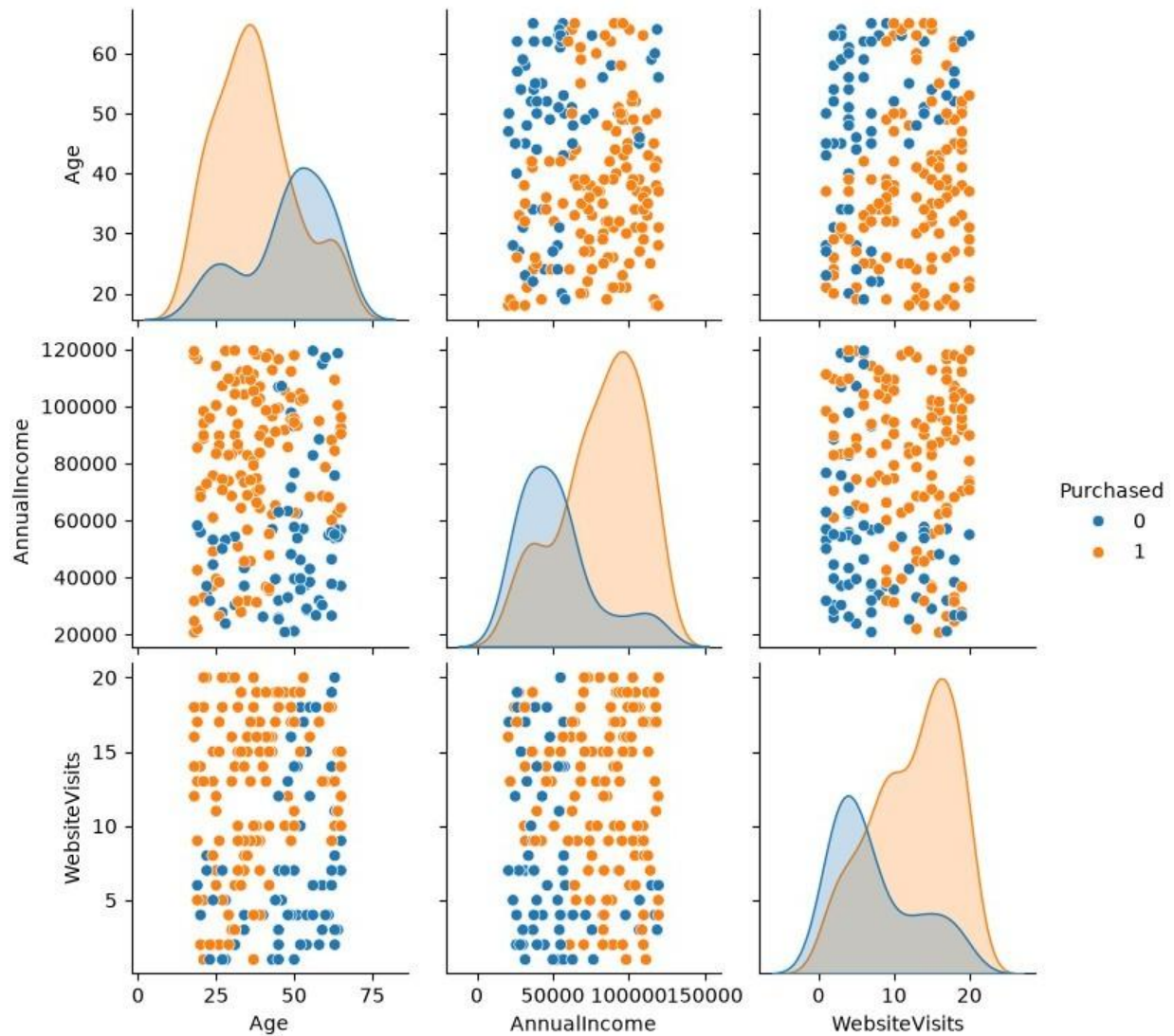
```
plt.hist(df['AnnualIncome'], bins=10)
plt.title("Annual Income Distribution")
plt.xlabel("Annual Income")
plt.ylabel("Frequency")
plt.show()
```



```
plt.hist(df['WebsiteVisits'], bins=10)
plt.title("Website Visits Distribution")
plt.xlabel("Website Visits")
plt.ylabel("Frequency")
plt.show()
```



```
sns.pairplot(df, hue='Purchased')  
plt.show()
```



```
X = df[['Age', 'AnnualIncome', 'WebsiteVisits']]
```

```
y = df['Purchased']
```

```
X.head()
```

	Age	AnnualIncome	WebsiteVisits
0	25	114130	7
1	63	109312	10
2	22	33944	8
3	43	62122	16
4	24	44479	2

```
y.head()
```

0	1
1	1

```

2     0
3     1
4     0
Name: Purchased, dtype: int64

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)

(160, 3)
(40, 3)
(160,)
(40,)

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

LogisticRegression(max_iter=1000)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy =", accuracy)

Accuracy = 0.875

cm = confusion_matrix(y_test, y_pred)

print(cm)

[[ 7  4]
 [ 1 28]]

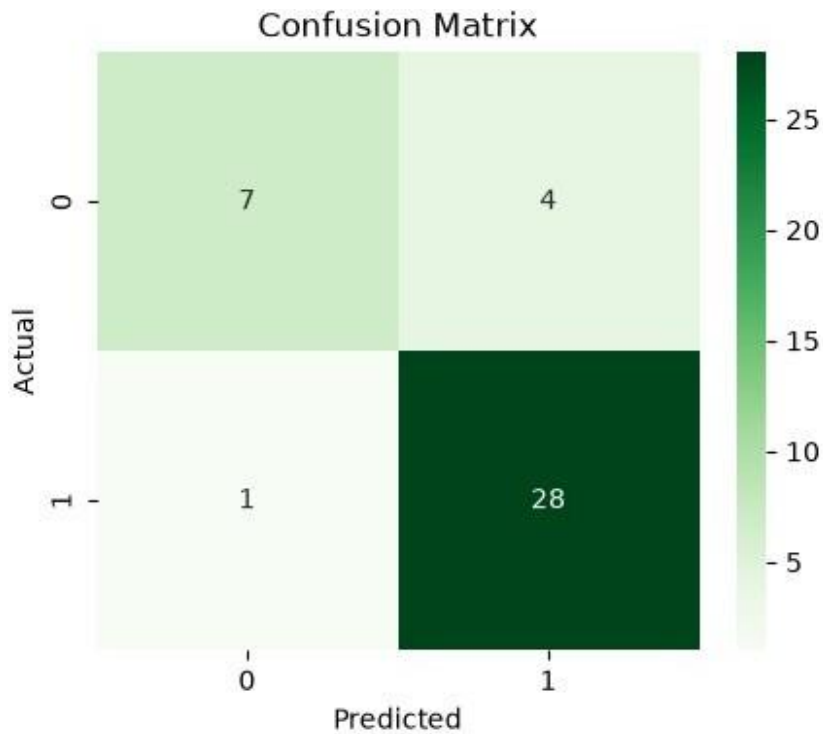
plt.figure(figsize=(5,4))

sns.heatmap(cm,
             annot=True,
             fmt='d',
             cmap='Greens')

plt.xlabel("Predicted")

```

```
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.88	0.64	0.74	11
1	0.88	0.97	0.92	29
accuracy			0.88	40
macro avg	0.88	0.80	0.83	40
weighted avg	0.88	0.88	0.87	40

```
print("Coefficients:")
```

```
print(model.coef_)
```

```
Coefficients:
[[-1.38643716e-01  7.99259375e-05  4.03517512e-01]]
```

```

print("Intercept:")

print(model.intercept_)

Intercept:
[-2.59644069]

customer = pd.DataFrame({
    'Age': [30],
    'AnnualIncome': [70000],
    'WebsiteVisits': [15]
})

prediction = model.predict(customer)

print("Prediction:", prediction)

Prediction: [1]

if prediction[0] == 1:
    print("Customer is likely to purchase the product.")
else:
    print("Customer is not likely to purchase the product.")

Customer is likely to purchase the product.

probability = model.predict_proba(customer)

print("Prediction Probability:")

print(probability)

Prediction Probability:
[[0.00745175 0.99254825]]

results = pd.DataFrame({
    'Actual': y_test,
    'Predicted': y_pred
})

results.head()

```

	Actual	Predicted
95	0	0
15	1	1
30	1	1
158	1	1
128	1	1

```

results.to_csv("Customer_Purchase_Predictions.csv", index=False)

print("Predictions saved successfully.")

```

Predictions saved successfully.